claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_b43bcbe7-b94\agent-a3cbcd5d646ec1c47.jsonl`
- SHA-256: `4f89876f223c233e82689316c8790177c93bd6cc7d8009fd0dafd68d22e3855e`
- Source modified: `2026-07-02T03:20:49+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b43bcbe7-b94`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T03:16:59.070Z)

```
You are an adversarial verifier in a tech-debt audit of the multi-repo workspace at
C:/Users/avidu/Projects/khelsutra-guru (repos: badminton-highlight-indexer, khelsutra, sports-
data-collector, rally-annotator, sports-obsreport; non-git dirs 'Annotation Setup', khelsutra-
screencast). Today is 2026-07-02. All repos are at origin head.
Default stance: REFUTE. A round-1 finder claimed (UNVERIFIED):
REPO: badminton-highlight-indexer
CLAIM: No timeout on any serving-path ffmpeg subprocess ? one hung ffmpeg permanently wedges the
single-worker job queue
INVESTIGATION HINTS: grep subprocess/ffmpeg call sites in backend/ (stitcher/compiler.py,
proxies, uploads, gemini extraction); check for timeout= args and the single-worker executor in
backend/api/job_worker.py. Failure scenario must be concrete: which serving path, what wedges.
VERIFY against the CURRENT working tree: (1) reproduce the defect/debt from actual code at
file:line; (2) check git log for a fix that already landed; (3) construct the concrete failure
scenario or downgrade severity; (4) propose the correctly-scoped fix with effort S/M/L; (5) flag
OWNER_GATED if it needs owner decisions (real cloud spend, product/strategy, counsel, Cloudflare
dashboard).
HARD RULES: READ-ONLY ? no file modifications, no installs (pip download/index checks and
WebFetch to pypi.org are OK), no test-suite runs. gh read-only OK. sqlite reads OK. CONFIRMED
requires you personally observed the defect in current code/data; cite file:line for every
claim.
```

2. user (2026-07-02T03:17:07.755Z)

```
<tool_use_error>Path does not exist: C:/Users/avidu/Projects/badminton-highlight-
indexer/backend. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer.</tool_use_error>
```

3. user (2026-07-02T03:17:08.492Z)

```
<tool_use_error>Path does not exist: C:/Users/avidu/Projects/badminton-highlight-
indexer/backend. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer.</tool_use_error>
```

4. user (2026-07-02T03:17:15.748Z)

```
output\nightly_regression\LATEST_STATUS.txt
.claude\scheduled_tasks.lock
```

5. user (2026-07-02T03:17:16.509Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\job_worker.py
```

6. user (2026-07-02T03:17:22.878Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:382:        out = subprocess.check_output(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:606:    rc = subprocess.run(cmd).returncode
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:1912:
subprocess.run([sys.executable, doctor_path])
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\annotation_proxy.py:31:        out = subprocess.check_output(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\annotation_proxy.py:81:        subprocess.run(cmd, check=True,
capture_output=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\audio\hit_detector.py:53:    subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\tools\rally_slicer.py:117:        out = subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\tools\rally_slicer.py:185:            subprocess.run(cmd, capture_output=True,
check=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:115:        return subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:129:        return subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\video.py:18:
subprocess.check_output(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\video.py:75:
subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py:43:            result = subprocess.run(cmd,
capture_output=True, text=True, check=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:166:                subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\stitcher\compiler.py:130:            result = subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\stitcher\compiler.py:321:                subprocess.run(cmd,
capture_output=True, check=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\stitcher\compiler.py:494:                subprocess.run(concat_cmd,
capture_output=True, check=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:69:        out = subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:123:        out = subprocess.run(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py:35:            result =
subprocess.run(cmd, capture_output=True, text=True, check=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py:15:        result = subprocess.run(
```

7. user (2026-07-02T03:17:24.062Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:203:def
_open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:212:
deadline = _time.monotonic() + timeout
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:215:
with urllib.request.urlopen(health_url, timeout=2) as r:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:1599:    can
run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it synchronously returned a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:1811:    a
failed sweep is logged and swallowed. The Gemini client is built with an HTTP timeout so
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\base.py:36:
(0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout or an
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:140:    poll exhaust its budget ? ``PolicyTimeout``. Catch it
```

```
and return a CLEAN rc 4 (remote dispatch
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:141:    timeout/failure, distinct from the local 2/3) rather
than letting a raw traceback escape."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:142:    from backend.infrastructure.execution_policy import
PolicyTimeout
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:152:    except PolicyTimeout as e:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:55:
timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:85:
timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:105:
# 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical to the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:4:- ``run_bounded`` ? a HARD per-attempt
timeout (``op_timeout_sec``, the hang-killer) + retry-with-backoff
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:41:class PolicyTimeout(TimeoutError):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:42:    """An op hung past ``op_timeout_sec``
or never became ready within ``max_wait_sec``."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:51:    op_timeout_sec: float = (
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:57:    on_timeout: str = "reschedule"  #
reschedule | partial | fail  (consumed by P2/P3)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:91:    """Run ``fn()`` under a hard per-
attempt timeout, retrying FAILURES and HANGS with backoff.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:93:    Returns ``fn``'s value on success.
Raises ``PolicyTimeout`` if the final attempt hung, or re-raises the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:110:            ) as e:  # relay ANY error
(incl. timeouts) to the caller thread
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:116:        if
done.wait(timeout=policy.op_timeout_sec):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:133:            last_exc = PolicyTimeout(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:134:                f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:141:                policy.op_timeout_sec,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:162:    ``PolicyTimeout`` after
``max_wait_sec``. ``check`` returns ``None`` while the op is still in progress.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:187:            raise PolicyTimeout(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\auth.py:48:
with urllib.request.urlopen(url, timeout=5) as resp:  # noqa: S310 (https team domain)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:46:        # busy_timeout: wait (not fail) on a
locked DB ? needed once the async job model
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:48:        conn = sqlite3.connect(self.db_path,
timeout=30.0)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:52:        # WAL allows concurrent readers during a
write; busy_timeout bounds lock waits.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:53:        conn.execute("PRAGMA busy_timeout =
```

```
30000")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\static\app.js:117:               await new Promise(res => setTimeout(res,
3000));
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\static\app.js:374:           // ffmpeg stitch can run minutes ? longer than
the edge/proxy timeout ? so it can't block
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\static\app.js:406:               await new Promise(resolve =>
setTimeout(resolve, 3000));
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\job_worker.py:35:#   - ExecutionPolicy op_timeout_sec kills hung
generate/processing calls
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\job_worker.py:36:#   - detector timeout_sec kills hung WSL/GPU calls
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\job_worker.py:38:# A future slice should add a per-job wall-clock timeout
(the executor itself
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\job_worker.py:39:# cannot enforce timeouts on the Thread).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:105:    and ``timeout_sec`` attributes
(TrackNetConfig and WasbConfig both qualify).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:106:    ``timeout_sec == 0`` maps to None = wait
forever (configs default 1800s).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:116:            argv, capture_output=True, text=True,
timeout=(self.cfg.timeout_sec or None)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:130:        except (subprocess.TimeoutExpired,
OSError):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:173:        except (subprocess.TimeoutExpired,
OSError) as e:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\base.py:189:        except (subprocess.TimeoutExpired,
OSError):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:81:    timeout_sec: int = 1800  # 30
min; a hung GPU call is killed (0 = no timeout)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:109:
timeout_sec=w.timeout_sec,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:134:
timeout=(self.cfg.timeout_sec or None),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:140:        (missing python / timeout /
import error) is treated as 'no CUDA'."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:143:        except (FileNotFoundError,
subprocess.TimeoutExpired):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:207:        except
subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:294:        except
subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py:296:                "WASB native
inference timed out after %ss.", self.cfg.timeout_sec
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:13:    PolicyTimeout,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
```

indexer\backend\providers\gemini.py:31:# Per-request HTTP timeout (ms) for the Gemini client ?
bounds list/upload/delete so a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:34:_HTTP_TIMEOUT_MS = 30_000
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:48:
http_options=types.HttpOptions(timeout=_HTTP_TIMEOUT_MS),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:70:   race a concurrent worker mid-upload). Bounded by
the client HTTP timeout. Never raises;
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:184:   except PolicyTimeout:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:211:   # 3. Call Gemini under a HARD per-attempt
timeout (op_timeout_sec ? the hang-killer that was
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:219:   op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:241:   except Exception as e: # PolicyTimeout (hang)
or the last 503 ? both end here
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:47:   timeout_sec: int = 1800 # 30 min;
kills a hung call (0 = no timeout)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:65:   timeout_sec=tn.timeout_sec,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:122:   except
subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:204:   except
subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:206:   "TrackNet inference
timed out after %ss.", self.cfg.timeout_sec
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:248:   except
subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:59:def gpu_stats(timeout: float = 4.0) ->
Optional[Dict[str, Any]]:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:73:   timeout=timeout,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:127:   timeout=4.0,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py:50:   timeout_sec: int = 1800 # 30 min; a
hung GPU/conda call is killed (0 = no timeout)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py:74:   timeout_sec=w.timeout_sec,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py:96:   except subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py:113:   except subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py:190:   except subprocess.TimeoutExpired:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py:191:   logger.error("WASB inference
timed out after %ss.", self.cfg.timeout_sec)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-
ukuS9wxa.js:25:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-
ukuS9wxa.js:48:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-
ukuS9wxa.js:49:[Omitted long matching line]

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-
ukuS9wxa.js:60:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:331:        # generate() give-ups (per-attempt
timeout / 503 bursts exhausting retries) are the SAME
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:341:
f"{generate_fails}/{num_chunks} chunks FAILED generate (timeout/503 after retries) ? "
```

8. assistant (2026-07-02T03:17:39.773Z)

The real repo is under `khelsutra-guru`. Let me read the job worker and the ffmpeg call sites.

9. user (2026-07-02T03:17:40.151Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import logging
20      import threading
21      import traceback
22      from concurrent.futures import ThreadPoolExecutor
23      from typing import Any, Dict, Optional
24
25      logger = logging.getLogger(__name__)
26
27      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
28      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
29      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
30      # already parallelize their AI handoff internally via their own pools.
31      #
32      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
33      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
34      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
35      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
36      #   - detector timeout_sec kills hung WSL/GPU calls
37      #   - Cloud Run poll has a max_wait_sec bound
38      # A future slice should add a per-job wall-clock timeout (the executor itself
39      # cannot enforce timeouts on the Thread).
40      _job_executor: Optional[ThreadPoolExecutor] = None
41
42
43      def _get_executor() -> ThreadPoolExecutor:
44          """Lazy module-global executor; tests monkeypatch this for inline execution."""
45          global _job_executor
46          if _job_executor is None:
47              _job_executor = ThreadPoolExecutor(
```

```
                max_workers=1, thread_name_prefix="jobworker"
            )
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main

        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
            # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
            # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
            # the job row for analytics). NULL ? server default = pre-picker behaviour.
            compute=job.get("compute"),
        )


    def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main

        job_id = job["id"]
        # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
        # original /api/process pipeline). Both record their terminal state the same way;
only process
```

```
        # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
        is_compile = job.get("kind") == "compile"
        try:
            if is_compile:
                result = _main._execute_compile(
                    config,
                    db,
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
                    db,
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
```

```
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main

        ran = 0
```

```
        while True:
            job = db.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job, config, db)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting(config, db)
        return ran


    def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
        """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
        self-scheduled job resumes with no further client submit. The drain itself re-checks
        due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
        import backend.main as _main

        try:
            delay = db.seconds_until_next_due()
        except Exception as e:  # pragma: no cover - defensive; never wedge the worker
            logger.error(f"Could not compute next-due delay: {e}")
            return
        if delay is None:
            return  # nothing waiting
        # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
        _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)


    def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
        """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
        scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
        monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
        import backend.main as _main

        timer = threading.Timer(
            delay_sec,
            lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
        )
        timer.daemon = True
        timer.start()


    #: A parked job further out than this is re-checked by a capped wake rather than one
long
    #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
    _MAX_DRAIN_WAKE_SEC = 60.0

```

10. user (2026-07-02T03:17:53.136Z)

```
        ) -> List[Tuple[float, float]]:
            """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.

            A highlight reel must never replay the same footage. The same rally can land in
            the segment list more than once ? e.g. two detector runs accumulated for one
            video (add_video is an UPSERT, so re-running without a prune leaves both sets),
```

```
or
            chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
            twice. Merging any range that overlaps or abuts the previous one collapses true
            duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
            Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
            """
            parsed = []
            for raw_s, raw_e in segments:
                s = VideoCompiler._parse_time(raw_s)
                e = VideoCompiler._parse_time(raw_e)
                if e > s:
                    parsed.append((s, e))
            parsed.sort()
            merged: List[Tuple[float, float]] = []
            for s, e in parsed:
                if merged and s <= merged[-1][1]:  # overlaps/abuts previous -> extend it
                    merged[-1] = (merged[-1][0], max(merged[-1][1], e))
                else:
                    merged.append((s, e))
            return merged

        def _get_video_duration(self, video_path: str) -> float:
            """Probes the video file to get its total duration in seconds.
            Returns 0.0 if it cannot be determined."""
            try:
                result = subprocess.run(
                    [
                        ffprobe_bin(),
                        "-v",
                        "error",
                        "-show_entries",
                        "format=duration",
                        "-of",
                        "default=noprint_wrappers=1:nokey=1",
                        video_path,
                    ],
                    capture_output=True,
                    text=True,
                    check=True,
                )
                return float(result.stdout.strip())
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(
                    f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
                )
                logo_path = ""
            if not text and not logo_path:
                return None

```

```
169             margin = int(wm.get("margin", 24))
170             opacity = float(wm.get("opacity", 0.85))
171             position = str(wm.get("position", "bottom-right"))
172             # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
173             dt_xy = {
174                 "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
175                 "bottom-left": f"x={margin}:y=h-th-{margin}",
176                 "top-right": f"x=w-tw-{margin}:y={margin}",
177                 "top-left": f"x={margin}:y={margin}",
178             }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
179             ov_xy = {
```

11. user (2026-07-02T03:17:54.139Z)

```
290                 ffmpeg_bin(),
291                 "-y",
292                 "-ss",
293                 str(round(padded_start, 3)),
294                 "-to",
295                 str(round(padded_end, 3)),
296                 "-i",
297                 raw_video_path,
298                 "-c:v",
299                 "libx264",
300                 "-preset",
301                 "superfast",
302                 "-crf",
303                 "22",
304                 "-c:a",
305                 "aac",
306                 "-b:a",
307                 "128k",
308             ]
309             vf_args = ["-vf", watermark_filter] if watermark_filter else []
310
311             # Try with the watermark first; if that encode fails (e.g. a bad font/logo
312             # path), retry once WITHOUT it so a branding misconfig can never nuke the
313             # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
314             attempts = [vf_args, []] if vf_args else [[]]
315             produced = False
316             last_exc: Optional[subprocess.CalledProcessError] = None
317             cmd = base_cmd + [clip_path]
318             for attempt_vf in attempts:
319                 cmd = base_cmd + attempt_vf + [clip_path]
320                 try:
321                     subprocess.run(cmd, capture_output=True, check=True)
322                 except subprocess.CalledProcessError as e:
323                     last_exc = e
324                     if attempt_vf and vf_args:
325                         _err = (
326                             e.stderr.decode(errors="replace").strip()[-300:]
327                             if getattr(e, "stderr", None)
328                             else ""
329                         )
330                         logger.warning(
331                             f"{segment_label}: watermark encode failed; retrying without it.
"
332                             f"ffmpeg: {_err or '(no stderr captured)'}"
333                         )
334                         continue
335                     break
336                 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
337                     produced = True
338                     if not attempt_vf and vf_args:
```

```
339                         logger.warning(
340                             f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
341                         )
342                     break
343                 last_exc = None  # ffmpeg succeeded but produced nothing
344                 break
345
346             if produced:
347                 return clip_path, None
348             elif last_exc is not None:
349                 stderr_msg = last_exc.stderr.decode(errors="replace").strip()
```

12. user (2026-07-02T03:18:01.182Z)

```
64:from backend.pipeline.stitcher.compiler import VideoCompiler
1644:def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1670:    compiler = VideoCompiler(
1942:        compiler = VideoCompiler(
```

13. user (2026-07-02T03:18:06.853Z)

```
1590            )
1591        return video, kept, out_name
1592
1593
1594    @app.post("/api/compile", status_code=202)
1595    def compile_highlights(req: CompileRequest, request: Request):
1596        """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).
1597
1598        Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
1599        can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
1600        Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
1601        got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
1602        fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
1603        {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
1604        # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
1605        try:
1606            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1607        except edge_auth.EdgeAuthError as e:
1608            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1609        try:
1610            video, _kept, out_name = _resolve_compile(
1611                request.app.state.db,
1612                request.app.state.config,
1613                req.video_id,
1614                req.segment_ids,
1615                req.output_filename,
1616            )
1617        except _CompileError as e:
1618            raise HTTPException(status_code=e.status_code, detail=e.detail) from e
1619
1620        job_id = uuid.uuid4().hex
1621        # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
1622        # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
```

```
        if not request.app.state.db.create_compile_job(
            job_id,
            req.video_id,
            video["filepath"],
            req.segment_ids,
            out_name,
            locale=req.locale,
            owner_id=owner_id,
        ):
            raise HTTPException(
                status_code=500, detail="Failed to persist compile job record."
            )
        _get_executor().submit(
            _drain_due_jobs, request.app.state.config, request.app.state.db
        )
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
        and return the {status, filepath, download_url} result the SPA polls for. Returns a
        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
        so the SPA always gets a clean verdict instead of a worker-crash error."""
        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        compiler = VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        notify("stitching")
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
```

```
video can't open gs://.
1683            local_src = storage.acquire_local_input(
1684                video["filepath"], getattr(cfg, "storage", None)
1685            )
1686            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1687        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1688            logger.error(
1689                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1690            )
1691            return {
1692                "status": "failed",
1693                "message": f"Highlight stitching failed: {e}",
1694                "video_id": video_id,
1695            }
1696
1697        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1698        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1699        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1700        notify("finished")
1701        return {
1702            "status": "success",
1703            "filepath": out_path,
1704            "download_url": download_url,
1705            "video_id": video_id,
1706        }
1707
1708
1709    # --- CLI Debug Utility & Orchestration ---
1710    def run_cli_diagnose_clip(video_path: str, timestamp: float):
1711        """CLI debugging utility to examine segment properties around a timestamp."""
1712        print("=" * 60)
1713        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
1714        print("=" * 60)
1715
1716        cfg = load_config()  # legacy wrapper still works, returns dict for now
1717
1718        # 1. Validation check diagnostics
1719        print("[DIAGNOSTIC] Running validation framework...")
```

14. assistant (2026-07-02T03:18:20.354Z)

`_execute_compile` calls `compiler.compile_highlights` directly at main.py:1686 with no timeout wrapper. Now let me check the process-path ffmpeg sites and any run_bounded wrapping.

15. user (2026-07-02T03:18:21.739Z)

```
140                     f"DEBUG FEATURE: Trimming video to first {debug_duration} seconds for
fast processing..."
141                 )
142                 base = output_base(self.config)
143                 temp_trimmed_path = os.path.join(base, f"temp_trimmed_{video_id}.mp4")
144                 os.makedirs(base, exist_ok=True)
145                 if os.path.exists(temp_trimmed_path):
146                     try:
147                         os.remove(temp_trimmed_path)
148                     except Exception:
149                         pass
150
151                 cmd = [
152                     ffmpeg_bin(),
153                     "-y",
154                     "-ss",
155                     "0",
```

```
                    "-t",
                    str(debug_duration),
                    "-i",
                    video_path,
                    "-c",
                    "copy",
                    temp_trimmed_path,
                ]
                logger.debug(f"Running command: {' '.join(cmd)}")
                try:
                    subprocess.run(
                        cmd,
                        check=True,
                        stdout=subprocess.DEVNULL,
                        stderr=subprocess.DEVNULL,
                    )
                    logger.info(
                        f"Trimmed video successfully created at: {temp_trimmed_path}"
                    )
                    video_to_upload = temp_trimmed_path
                    is_trimmed = True
                except Exception as e:
                    logger.warning(
                        f"Failed to trim video using ffmpeg: {e}. Falling back to full
video."
                    )

            # Helper to clean up local temp files
            def cleanup_temp_files():
                if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
                    except Exception:
                        pass

```

16. user (2026-07-02T03:18:22.236Z)

```
import logging
import subprocess
from typing import Optional

from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
    ffmpeg_bin,
    ffprobe_bin,
)

logger = logging.getLogger(__name__)


def get_video_duration(video_path: str) -> float:
    """Returns the exact duration of a video file in seconds using ffprobe.
    Returns 0.0 on failure."""
    try:
        duration_output = (
            subprocess.check_output(
                [
                    ffprobe_bin(),
                    "-v",
                    "error",
                    "-show_entries",
                    "format=duration",
                    "-of",
                    "default=noprint_wrappers=1:nokey=1",
                    video_path,
```

```
                    ]
                )
                .decode()
                .strip()
            )
            return float(duration_output)
        except Exception as e:
            logger.error(f"Could not detect video duration for {video_path}: {e}")
            return 0.0


    def extract_video_chunk(
        video_path: str,
        start_time: float,
        duration: float,
        output_path: str,
        reencode: bool = False,
        scale_width: Optional[int] = None,
    ) -> bool:
        """Extracts a chunk from a video file.

        Args:
            reencode: If True, re-encodes with libx264 (slower but frame-accurate).
                      If False, uses stream-copy (fast but cuts at nearest keyframe).
                      Use True for short clips where keyframe drift matters.
            scale_width: If set (requires reencode), downscale to this width (height auto,
                      even). Useful for high-res sources headed to an API that analyses at
                      low res anyway ? shrinks uploads with no analysis loss.
        """
        cmd = [
            ffmpeg_bin(),
            "-y",
            "-ss",
            str(start_time),
            "-t",
            str(duration),
            "-i",
            video_path,
        ]
        if reencode:
            if scale_width:
                cmd += ["-vf", f"scale={int(scale_width)}:-2"]
            cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
        else:
            cmd += ["-c", "copy"]
        cmd.append(output_path)
        try:
            subprocess.run(
                cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
            )
            return True
        except Exception as e:
            logger.error(
                f"Failed to extract chunk {start_time}s - {start_time + duration}s: {e}"
            )
            return False

```

17. user (2026-07-02T03:18:23.116Z)

```
      """Frame-preserving annotation proxy (STUDIO_MEDIA_LIFECYCLE.md D4).

      A big 4K master is heavy to decode + scrub in a browser. For in-UI annotation we serve a
downscaled
      proxy that keeps the SAME timeline (fps + duration) ? so a rally marked at t seconds on
```

```
the proxy is
5       t seconds on the master, and the labels still key to the master's file MD5. Only
resolution is
6       dropped; duration parity is VERIFIED after the transcode, and a proxy whose timeline
drifted is
7       discarded (the caller falls back to serving the source). Best-effort + stdlib-only;
never raises.
8       """
9       from __future__ import annotations
10
11      import logging
12      import os
13      import subprocess
14      import tempfile
15
16      from backend.utils.ffmpeg import ffmpeg_available, ffmpeg_bin, ffprobe_bin
17      from backend.utils.video import get_video_duration
18
19      logger = logging.getLogger(__name__)
20
21      #: Max proxy height (px). A source already at/below this needs no proxy (serving it is
already light).
22      DEFAULT_MAX_HEIGHT = 720
23      #: Duration must match the source within this (s), else the timeline drifted ? discard
(protects the
24      #: second-based marks the annotator produces).
25      _DURATION_TOLERANCE_S = 0.5
26
27
28      def probe_height(path: str) -> int:
29          """The video's height in px, or 0 on failure."""
30          try:
31              out = subprocess.check_output(
32                  [ffprobe_bin(), "-v", "error", "-select_streams", "v:0", "-show_entries",
33                   "stream=height", "-of", "default=noprint_wrappers=1:nokey=1", path],
34                  text=True,
35              ).strip().splitlines()
36              return int(out[0]) if out and out[0].strip() else 0
37          except Exception as e:  # noqa: BLE001 - probe is best-effort
38              logger.warning("proxy: could not probe height for %s: %s", path, e)
39              return 0
40
41
42      def needs_proxy(src: str, max_height: int = DEFAULT_MAX_HEIGHT) -> bool:
43          """True when ``src`` is tall enough that a downscaled proxy is worth generating."""
44          return probe_height(src) > max_height
45
46
47      def _cleanup(path: str) -> None:
48          try:
49              os.remove(path)
50          except OSError:
51              pass
52
53
54      def make_annotation_proxy(src: str, dst: str, max_height: int = DEFAULT_MAX_HEIGHT) ->
bool:
55          """Transcode ``src`` ? a downscaled, timeline-preserving proxy at ``dst`` (written
atomically).
56          Returns True on a verified proxy, False otherwise (caller serves the source). Never
raises.
57
58          Only the resolution is scaled (``scale=-2:H``); fps/duration are untouched and
duration parity is
59          verified, so second-based marks transfer to the master unchanged. Audio is kept (the
```

```
shuttle
60          'thwack' helps the rater judge rally boundaries)."""
61          if not ffmpeg_available():
62              logger.info("proxy: ffmpeg unavailable; serving source (%s)", src)
63              return False
64          if not os.path.isfile(src):
65              return False
66          src_dur = get_video_duration(src)
67          dst_dir = os.path.dirname(dst) or "."
68          os.makedirs(dst_dir, exist_ok=True)
69          fd, tmp = tempfile.mkstemp(dir=dst_dir, prefix=".proxy-", suffix=".mp4")
70          os.close(fd)
71          cmd = [
72              ffmpeg_bin(), "-y", "-i", src,
73              "-vf", f"scale=-2:{max_height}",
74              "-c:v", "libx264", "-preset", "veryfast", "-crf", "28", "-pix_fmt", "yuv420p",
75              "-g", "48",  # denser keyframes ? snappy scrubbing
76              "-c:a", "aac", "-b:a", "96k",
77              "-movflags", "+faststart",
78              tmp,
79          ]
80          try:
81              subprocess.run(cmd, check=True, capture_output=True)
82          except Exception as e:  # noqa: BLE001 - a transcode failure is non-fatal; fall back
to the source
83              logger.warning("proxy: ffmpeg failed for %s: %s", src, e)
84              _cleanup(tmp)
85              return False
86          proxy_dur = get_video_duration(tmp)
87          if src_dur > 0 and abs(proxy_dur - src_dur) > _DURATION_TOLERANCE_S:
88              logger.warning("proxy: duration parity off for %s (src=%.3fs proxy=%.3fs);
discarding",
89                             src, src_dur, proxy_dur)
90              _cleanup(tmp)
91              return False
92          try:
93              os.replace(tmp, dst)
94          except OSError as e:
95              logger.warning("proxy: could not place proxy at %s: %s", dst, e)
96              _cleanup(tmp)
97              return False
98          logger.info("proxy: wrote annotation proxy %s (%.1fs)", dst, proxy_dur)
99          return True
100
```

18. user (2026-07-02T03:18:40.690Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:4:- ``run_bounded`` ? a HARD per-attempt
timeout (``op_timeout_sec``, the hang-killer) + retry-with-backoff
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:83:def run_bounded(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:15:    run_bounded,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:233:            response = run_bounded(
```

19. user (2026-07-02T03:18:41.997Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:612:
from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:613:
if not needs_proxy(src_local):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:615:
make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4"))
```

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\annotation_proxy.py:42:def needs_proxy(src: str, max_height: int =
DEFAULT_MAX_HEIGHT) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\annotation_proxy.py:54:def make_annotation_proxy(src: str, dst: str,
max_height: int = DEFAULT_MAX_HEIGHT) -> bool:
```

20. user (2026-07-02T03:18:43.249Z)

```
263:        # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
291:        # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
842:    def mark_job_running(self, job_id: str) -> bool:
844:            "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
847:            f"Failed to mark job {job_id} running",
1062:        worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
1094:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
1096:        flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
1110:                    "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
1122:    def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
1123:        """Crash recovery: any job still queued/running from a previous process is dead
1131:                    "WHERE status IN ('queued', 'running')",
1137:            logger.error(f"Failed to sweep stale jobs: {e}")
```

21. user (2026-07-02T03:18:53.096Z)

```
575             name = safe_output_filename(filename)
576         except ValueError as e:
577             raise HTTPException(status_code=400, detail=str(e)) from e
578         base = _reels_dir(request.app.state.config)
579         path = os.path.join(base, name)
580         try:
581             path = resolve_under_root(path, base)
582         except ValueError as e:
583             raise HTTPException(status_code=400, detail=str(e)) from e
584         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
585             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
586         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
587         return FileResponse(path, media_type=media, filename=name)
588
589
590     # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
----------------
591
592
593     # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
/api/source
594     # don't each kick off a transcode.
595     _proxy_inflight: "set[str]" = set()
596     _proxy_lock = threading.Lock()
597
598
599     def _proxies_dir(cfg) -> str:
600         return os.path.join(output_base(cfg), "proxies")
601
602
603     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
604         """Background, best-effort: generate the annotation proxy ONCE so subsequent
/api/source loads
605         serve the lighter, seekable copy. Deduped; never blocks the request; the source
keeps serving if
606         it fails."""
607         with _proxy_lock:
608             if video_id in _proxy_inflight:
```

```
609                 return
610             _proxy_inflight.add(video_id)
611         try:
612             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
613             if not needs_proxy(src_local):
614                 return  # already light enough ? no proxy worth making
615             make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
f"{video_id}.mp4"))
616         except Exception as e:  # noqa: BLE001 - warming is best-effort
617             logger.warning("proxy: warm failed for %s: %s", video_id, e)
618         finally:
619             with _proxy_lock:
620                 _proxy_inflight.discard(video_id)
621
622
623     @app.get("/api/source/{video_id}")
624     def get_source(video_id: str, request: Request):
625         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
626         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
627         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
628         background for next time ? non-blocking, so the first load is never delayed. Proxy
warming is
629         LOCAL-source only (remote-source proxying is a follow-up)."""
630         rec = request.app.state.db.get_video(video_id)
631         if not rec or not rec.get("filepath"):
632             raise HTTPException(status_code=404, detail="Video not found.")
633         cfg = request.app.state.config
634
635         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
636         try:
637             proxy_name = safe_output_filename(f"{video_id}.mp4")
638             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
_proxies_dir(cfg))
639         except ValueError:
640             proxy_path = ""
641         if proxy_path and os.path.isfile(proxy_path):
642             return FileResponse(proxy_path, media_type="video/mp4")
643
644         # 2) Otherwise resolve + serve the source.
645         try:
646             local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
None))
647             is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
None))
648         except ValueError as e:  # unsupported scheme etc. ? a clean client error, never a
500
649             raise HTTPException(status_code=400, detail=str(e)) from e
```

22. user (2026-07-02T03:18:54.069Z)

```
30
31      def extract_audio(
32          video_path: str, out_wav: Optional[str] = None, sr: int = 16000
33      ) -> str:
34          """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
35          Audio-only decode, so it's fast even for multi-GB video."""
36          if out_wav is None:
37              fd, out_wav = tempfile.mkstemp(suffix=".wav")
38              os.close(fd)
39          cmd = [
40              ffmpeg_bin(),
41              "-y",
```

```
42              "-i",
43              video_path,
44              "-vn",
45              "-ac",
46              "1",
47              "-ar",
48              str(sr),
49              "-c:a",
50              "pcm_s16le",
51              out_wav,
52          ]
53          subprocess.run(
54              cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
55          )
56          return out_wav
57
58
59      def load_wav(path: str) -> Tuple[np.ndarray, int]:
60          """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
61          with wave.open(path, "rb") as w:
62              sr, n, sw, ch = (
63                  w.getframerate(),
64                  w.getnframes(),
65                  w.getsampwidth(),
66                  w.getnchannels(),
67              )
68              raw = w.readframes(n)
69          if sw == 2:
70              a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
71          elif sw == 4:
72              a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
73          elif sw == 1:
74              a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
```

23. assistant (2026-07-02T03:19:06.491Z)

Let me check where `_warm_annotation_proxy` is invoked from, the second VideoCompiler use at main.py:1942, and git history for any landed fix.

24. user (2026-07-02T03:19:08.015Z)

```
599-def _proxies_dir(cfg) -> str:
600-    return os.path.join(output_base(cfg), "proxies")
601-
602-
603:def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
604-    """Background, best-effort: generate the annotation proxy ONCE so subsequent /api/source
loads
605-    serve the lighter, seekable copy. Deduped; never blocks the request; the source keeps
serving if
606-    it fails."""
607-    with _proxy_lock:
--
653-        raise HTTPException(status_code=404, detail="Source video file not found.")
654-
655-    # 3) Warm a proxy in the background for next time (local source only).
656-    if is_local:
657:        threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
658-
659-    media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
660-    return FileResponse(local, media_type=media)
661-
```

25. user (2026-07-02T03:19:08.800Z)

```
1925                out_name = os.path.basename(out_path)
1926            else:
1927                out_dir = resolve_output_dir(
1928                    args.output_dir
1929                )  # P0a: app-home aware (CWD-identical when unset)
1930                out_path = os.path.join(out_dir, out_filename)
1931                out_name = out_filename
1932
1933            os.makedirs(out_dir, exist_ok=True)
1934            print(
1935                f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1936            )
1937
1938            # global_config isn't initialized this early; load the typed config so the trim
1939            # clip carries the same configured stitching paddings + watermark as a real
1940            # highlight (watermark default-on).
1941            stitching = load_app_config().stitching
1942            compiler = VideoCompiler(
1943                out_dir,
1944                begin_padding=stitching.begin_padding,
1945                end_padding=stitching.end_padding,
1946                watermark=stitching.watermark,
1947            )
1948            try:
1949                res_path = compiler.compile_highlights(
1950                    args.video_path, [(args.trim_start, args.trim_end)], out_name
1951                )
1952                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1953            except Exception as e:
1954                print(f"[ERROR] Failed to compile trimmed segment: {e}")
1955            return
1956
1957        if args.debug_clip is not None:
1958            if not args.video_path:
1959                print("[ERROR] Please provide --video-path to debug a specific clip.")
1960                return
1961            run_cli_diagnose_clip(args.video_path, args.debug_clip)
1962            return
1963
1964        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
```

26. assistant (2026-07-02T03:19:18.162Z)

CLI path at 1942 ? not serving. Now let me trace `_execute_processing` to confirm which unbounded ffmpeg calls sit on the process-job path, and check git history for a landed fix.

27. user (2026-07-02T03:19:20.132Z)

```
1132:def _execute_processing(
1133-    config=None, db=None, req=None, progress=None
1134-) -> Dict[str, Any]:
1135-    """The full hash ? validate ? segment ? report pipeline for one video.
1136-
1137-    Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
1138-    module-level ``global_config``/``db_instance`` (backward-compatible with existing
1139-    tests that pass only ``req``). Route handlers always pass explicit params; older
1140-    test code that calls ``_execute_processing(req)`` still works.
1141-
1142-    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
1143-    Raises on unexpected internal errors ? the caller records those as a job failure
1144-    """
```

```
1145-    # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
1146-    # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
1147-    if config is not None and not isinstance(config, AppConfig):
1148-        # Old signature: _execute_processing(req, progress=None)
1149-        if req is None and db is not None and isinstance(db, ProcessRequest):
1150-            req = db
1151-            db = None
1152-        elif req is None and isinstance(config, ProcessRequest):
1153-            req = config
1154-            config = None
1155-    # Resolve config/db from module globals when not passed explicitly
1156-    if config is None:
1157-        config = global_config
1158-    if db is None:
1159-        db = db_instance
1160-
1161-    notify = progress or (lambda stage: None)
1162-
1163-    notify("hashing")
1164-    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
1165-    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166-    # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167-    # consumers (hash / validators / segmenter) use the resolved local path.
1168-    local_video = storage.acquire_local_input(
1169-        req.video_path, getattr(config, "storage", None)
1170-    )
1171-    video_id = compute_video_id(local_video)
1172-    was_reingest = db.get_video(video_id) is not None
1173-
1174-    # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175-    notify("validating")
1176-    logger.info(f"Running validations for {req.video_path}...")
1177-    val_results, val_context = registry.run_all_with_context(local_video, config)
1178-    failures = [r for r in val_results if not r.passed]
1179-
1180-    # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181-    default_seg = getattr(
1182-        getattr(config, "indexing", None), "default_segmenter", "motion"
1183-    )
1184-    seg_name = req.segmenter or default_seg
1185-    segmenter_actual = None
1186-    segmentation_ran = False
1187-    # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188-    # "in_process" once the local segmenter is reached, set on the response as "cloud_run"
by
1189-    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190-    compute_actual: Optional[str] = None
1191-    response: Optional[Dict[str, Any]] = None
1192-    try:
```

28. user (2026-07-02T03:19:22.095Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
```

```
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
24178a7 docs(job_worker): document single-executor O2 risk + existing mitigations (#374)
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
```

29. user (2026-07-02T03:19:55.811Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-2027-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-2028-    #
Crash recovery (#16): jobs left queued/running by a previous process are dead ?
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-2029-    #
the executor is in-process. Mark them failed so pollers see a terminal state.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:2030:
swept = db_instance.fail_stale_jobs()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-2031-    if
swept:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-2032-
logger.warning(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-2033-
f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed."
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py-1119-            logger.error(f"Failed to claim a due
job: {e}")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py-1120-            return None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py-1121-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:1122:    def fail_stale_jobs(self, reason: str =
"interrupted by server restart") -> int:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py-1123-        """Crash recovery: any job still
queued/running from a previous process is dead
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py-1124-        (the executor is in-process). Mark
failed so clients see a terminal state, not a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py-1125-        hang. Called once at startup. Returns
the number of jobs swept."""
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\single_instance.py-1-"""Single-instance file lock (PACKAGING_PLAN.md P0e).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\single_instance.py-2-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\single_instance.py:3:The serving model runs an IN-PROCESS job worker, and
``fail_stale_jobs()`` sweeps any job left
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\single_instance.py-4-``queued``/``running`` by a "previous" process to
``failed`` on startup. If a SECOND process opens the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\single_instance.py-5-same SQLite DB (a second ``indexer --server``, or a
CLI run while a server is up), its sweep would
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\single_instance.py-6-mark the FIRST process's in-flight jobs failed ?
corrupting live state. This module guards the DB
```

30. user (2026-07-02T03:19:56.339Z)

```
480                     ffmpeg_bin(),
481                     "-y",
482                     "-f",
```

```
                    "concat",
                    "-safe",
                    "0",
                    "-i",
                    list_file_path,
                    "-c",
                    "copy",
                    output_path,
                ]

                try:
                    subprocess.run(concat_cmd, capture_output=True, check=True)

                    # Final verification
                    if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
                        raise RuntimeError(
                            f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
                            f"but output file is missing or empty: {output_path}"
                        )

                    output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
                    logger.info(
                        f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
                    )
                    return output_path
                except subprocess.CalledProcessError as e:
                    stderr_msg = e.stderr.decode(errors="replace").strip()
```

31. user (2026-07-02T03:20:49.671Z)

```
Structured output provided successfully
```